

Pocket App — Proposal

Thông Tin Nhóm

Môn học: Mobile Programming

Nhóm: 03

Họ và Tên	MSSV
Huỳnh Gia Hân	23110019
Bùi Trần Tấn Phát	23110052
Nguyễn Nhật Phát	23110053

Tổng Quan Dự Án

Pocket là ứng dụng chia sẻ ảnh thời gian thực theo mô hình của Locket Widget — cho phép người dùng gửi ảnh tức thời đến màn hình chính (Home Screen Widget) của bạn bè và người thân. Ứng dụng tập trung vào kết nối thân mật, không theo dõi lượt thích hay follower, phù hợp với xu hướng mạng xã hội "anti-social" đang được Gen Z ưa chuộng.

Locket Widget được ra mắt vào tháng 1/2022 và nhanh chóng trở thành viral, đứng đầu bảng xếp hạng App Store chỉ nhờ một tính năng cốt lõi làm xuất sắc: hiển thị ảnh từ bạn bè trực tiếp lên widget màn hình chính. Đây là nền tảng để nhóm xây dựng Pocket với những cải tiến phù hợp hơn cho người dùng Việt Nam.

Phân Tích Locket Widget (Competitor Analysis)

Tính năng cốt lõi của Locket

Tính năng	Mô tả
Home Screen Widget	Hiển thị ảnh bạn bè real-time trên màn hình chính
Camera nhanh	Tap widget → mở camera → chụp → gửi trong 3 bước
Giới hạn 20 bạn	Chỉ kết nối với người thân thiết, không cần follower
Emoji reactions	Phản hồi ảnh bằng emoji, không đếm like công khai
Photo history	Lưu lịch sử tất cả ảnh đã gửi nhau
Chat trong app	Nhắn tin trực tiếp dưới ảnh

Điểm yếu của Locket cần Pocket cải thiện

- Chưa có tính năng AI filter / caption thông minh
- Không hỗ trợ video (chỉ ảnh tĩnh)
- Android từng gặp bugs và thiếu tính năng so với iOS

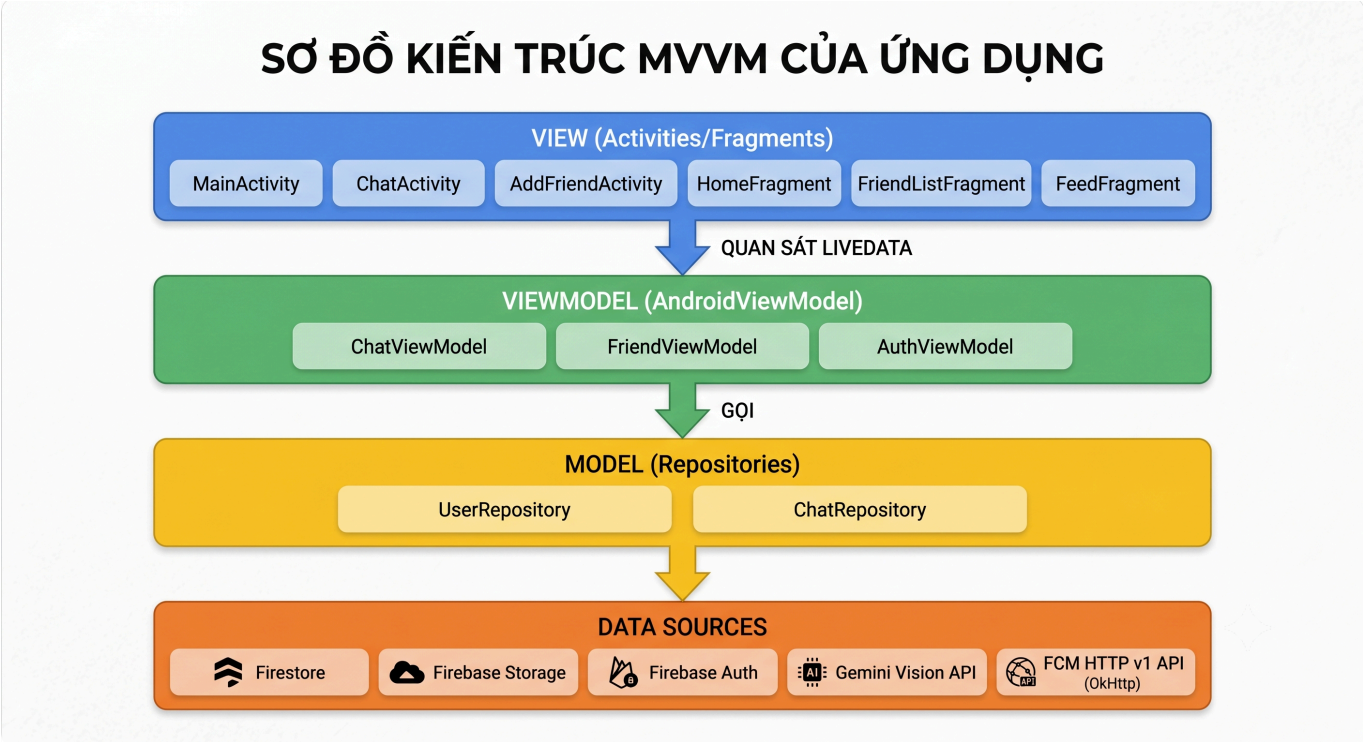
- Không có tùy chỉnh widget (chỉ 1 kiểu hiển thị)
- Notification không cá nhân hóa nội dung

Kiến Trúc Kỹ Thuật (Technical Architecture)

Tech Stack Thực Tế

Layer	Technology	Lý do chọn
Mobile Framework	Android Native (Java)	Kiến thức môn học, không cần cross-platform
Architecture Pattern	MVVM + ViewModel	Tách biệt UI/logic, lifecycle-aware, dễ maintain
Backend & Auth	Firebase Auth	Phone number auth (OTP), quản lý session
Realtime Database	Cloud Firestore	Realtime listener cho feed ảnh và chat
File Storage	Firebase Cloud Storage	Lưu trữ ảnh, trả về download URL
Push Notification	FCM Client-side (via OkHttp + FCM HTTP v1 API)	Gửi notification trực tiếp từ client — phù hợp cho demo/học, không cần Cloud Function
Image Loading	Glide	Cache và load ảnh hiệu quả, phổ biến trên Android
UI Components	Material Design 3	Bottom nav pill, CircleImageView, Snackbar
AI Caption	Gemini Vision API	Auto caption từ ảnh bằng tiếng Việt

Kiến trúc MVVM



Luồng Hoạt Động (User Flow)

Luồng 1: Onboarding & Đăng ký

```

Mở app
├─> Kiểm tra Firebase Auth session
│   ├──> Đã đăng nhập → vào thẳng MainActivity (skip login)
│   └─> Chưa đăng nhập → LoginActivity
│       └─> Nhập số điện thoại
│           └─> OtpActivity (xác thực OTP)
│               └─> Tạo profile (tên hiển thị, ảnh đại diện)
│                   └─> MainActivity (Home Screen)
  
```

Luồng 2: Gửi Ảnh (Core Flow)

```

Tap nút Camera ở bottom nav
├─> Mở CameraActivity
│   ├──> Chụp ảnh (front / back camera)
│   │   └─> [Optional] AI auto-caption gợi ý (Gemini Vision)
│   ├──> Chọn người nhận (từ danh sách bạn bè)
│   └─> Nhấn Send
│       ├──> Upload ảnh lên Firebase Storage
│       ├──> Lưu metadata vào Firestore /photos/{photoId}
│       └─> FcmHelper gửi notification đến người nhận (OkHttp → FCM HTTP v1 API)
  
```

Luồng 3: Nhận Ảnh (Receiver Flow)

```

Bạn gửi ảnh
├─> PocketMessagingService.onMessageReceived()
│   └─> Hiển thị local notification (NotificationCompat)
│       └─> Người nhận tap notification → mở ChatActivity / HomeFragment
│           └─> Xem ảnh full screen
│               ├──> React emoji
│               ├──> Reply chat
│               └─> Xem lịch sử ảnh
  
```

Luồng 4: Chat & Bạn bè

```

Từ FriendListFragment
├─> Tap nút Message trên friend item
│   └─> ChatActivity (với friendUid, friendName, friendAvatar)
│       └─> Real-time messages qua Firestore listener
  
```

```

└─> Gửi tin nhắn → lưu Firestore + gửi FCM notification
└─> Seen receipt khi đối phương mở chat

```

Các Tính Năng Chi Tiết (Feature Breakdown)

Module 1: Authentication

- Đăng nhập bằng số điện thoại (OTP via Firebase Auth)
- Quản lý session, **auto-login** (bỏ qua màn hình đăng nhập nếu đã có session)
- Đặt tên hiển thị, ảnh đại diện khi tạo profile lần đầu
- Đăng xuất từ ProfileFragment

Module 2: Friend System

- Tìm bạn qua số điện thoại (search trên Firestore)
- Gửi / nhận / chấp nhận / từ chối lời mời kết bạn
- **Ẩn số điện thoại** trong kết quả tìm kiếm và danh sách bạn — chỉ hiển thị tên
- Xóa bạn bè (remove friend)
- Danh sách bạn bè với pending requests section

Module 3: Camera & Photo Capture

- Camera screen với chuyển đổi camera trước/sau
- Chụp ảnh tức thì (không chọn từ gallery)
- Upload ảnh lên Firebase Cloud Storage
- Lưu metadata (sender, receiver, imageUrl, timestamp) vào Firestore

Module 4: Notification System

Cách hoạt động thực tế (Client-side FCM):

```

Người gửi nhắn Send
└─> FcmHelper.sendMessageNotification(receiverUid, senderName, message)
    └─> Lấy fcmToken của người nhận từ Firestore users/{receiverUid}
    └─> Tạo JSON payload FCM
    └─> OkHttp POST →
https://fcm.googleapis.com/v1/projects/{projectId}/messages:send
    └─> FCM gửi đến thiết bị người nhận
        └─> PocketMessagingService.onMessageReceived()
            └─> NotificationCompat hiển thị notification

```

- Notification hiển thị tên người gửi làm title, nội dung tin nhắn làm body
- Tap notification mở thẳng **ChatActivity** với đúng người gửi (PendingIntent)
- Token FCM được lưu vào Firestore khi đăng nhập hoặc token refresh
- Notification channel **pocket_messages** (importance HIGH) tạo trong **PocketApplication**

Lưu ý kỹ thuật: Phương pháp client-side FCM phù hợp cho môi trường học tập và demo. Trong production, nên dùng Firebase Cloud Functions (server-side trigger) để đảm bảo bảo mật và độ tin cậy cao hơn.

Module 5: Real-time Chat

- Nhắn tin 1-1 với bạn bè
- Real-time Firestore listener (`addSnapshotListener`) cập nhật tin nhắn tức thì
- Keyboard-aware layout (`WindowInsetsCompat`) tự điều chỉnh khi bàn phím xuất hiện
- Hiển thị avatar và tên người nhận trên toolbar
- Message bubbles phân biệt tin của mình và tin của đối phương

Module 6: Photo History (Memories)

- Timeline ảnh đã gửi/nhận theo thứ tự thời gian
- Xem full screen từng ảnh
- Phân trang để tối ưu performance

Module 7: Profile

- Xem và chỉnh sửa tên hiển thị
- Thay đổi ảnh đại diện (upload lên Firebase Storage)
- Đăng xuất

Tích Hợp AI

Pocket tích hợp **Gemini Vision API** để tạo caption tự động từ ảnh.

AI Auto-Caption

Cách hoạt động:

1. Người dùng chụp ảnh trong `CameraActivity`
2. Gọi Gemini Vision API với prompt tiếng Việt
3. Hiển thị gợi ý caption để user chọn hoặc bỏ qua
4. Caption gửi kèm ảnh cho bạn nhận

Ví dụ prompt gửi lên Gemini:

"Mô tả ngắn gọn bức ảnh này trong 1 câu bằng tiếng Việt, giọng điệu vui vẻ, thân thiện, phù hợp với bạn bè Gen Z."

Nhược điểm cần lưu ý:

- Gemini API free tier có rate limit → cache kết quả, chỉ gọi khi user chủ động nhấn "Gợi ý caption"
- Cần xử lý timeout và fallback khi API không phản hồi

Cấu Trúc Database (Firestore Schema)

```
/users/{uid}
- displayName: String
- phoneNumber: String      ← chỉ dùng để tìm kiếm, không hiển thị cho user khác
- avatarUrl: String
- fcmToken: String         ← cập nhật mỗi khi login hoặc token refresh
- friends: [uid1, uid2, ...]
- friendRequests: [uid1, ...]
- createdAt: Timestamp

/chats/{chatId}             ← chatId = uid1_uid2 (sorted alphabetically)
- participants: [uid1, uid2]
- lastMessage: String
- lastUpdated: Timestamp

/chats/{chatId}/messages/{msgId}
- senderId: String
- text: String
- type: String (text | photo_reply)
- createdAt: Timestamp
- seen: Boolean

/photos/{photoId}
- senderId: String
- receiverIds: [uid1, uid2]
- imageUrl: String
- caption: String (optional, AI-generated)
- createdAt: Timestamp
- seenBy: [uid1, uid2]
```

Các Dependencies Chính (build.gradle)

Dependency	Mục đích
firebase-auth	Phone OTP authentication
firebase-firestore	Realtime database
firebase-storage	Image storage
firebase-messaging	FCM token & receive
glide	Image loading & caching
circleimageview	Avatar hiển thị tròn
material	Material Design 3 components
okhttp3	HTTP client để gọi FCM HTTP v1 API

Dependency	Mục đích
Gemini Android SDK	AI caption từ ảnh

Lưu Ý Kỹ Thuật Quan Trọng

1. Notification — Client-side FCM

Thay vì dùng Firebase Cloud Function (yêu cầu Blaze plan trả phí), Pocket gửi notification trực tiếp từ Android client thông qua FCM HTTP v1 API:

```
// FcmHelper.java
public static void sendMessageNotification(
    String receiverUid, String senderName, String messageText) {
    // 1. Lấy fcmToken từ Firestore
    // 2. Build JSON payload
    // 3. POST bằng OkHttp → FCM HTTP v1 API
    // 4. Chạy trên background thread (ExecutorService)
}
```

Nhược điểm đã biết: Client không nên gọi thẳng FCM API vì lý do bảo mật. Người dùng khác có thể giả mạo notification nếu biết fcmToken. Trong production cần chuyển sang Cloud Function.

2. Firestore Security Rules

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /users/{uid} {
      allow read: if request.auth != null;           // bạn bè có thể đọc profile
      allow write: if request.auth.uid == uid;       // chỉ chính chủ mới sửa
    }
    match /chats/{chatId}/messages/{msgId} {
      allow read, write: if request.auth.uid in
        get(/databases/{database}/documents/chats/{chatId}).data.participants;
    }
  }
}
```

3. Ẩn Số Điện Thoại

Số điện thoại chỉ dùng để tìm kiếm trong `AddFriendActivity`. Sau khi hiển thị kết quả, `resultPhone.setVisibility(View.GONE)`. Trong `FriendListFragment`, phone TextView mặc định là `GONE`.

4. Auto-Login

Trong `LoginActivity.onCreate()`, kiểm tra `FirebaseAuth.getInstance().getCurrentUser() != null` trước `setContentView`. Nếu đã đăng nhập → `startActivity(MainActivity) + finish()`.

Rủi Ro & Giải Pháp

Rủi ro	Mức độ	Giải pháp
FCM client-side không an toàn	Trung bình	Chấp nhận cho demo học tập, ghi chú trong báo cáo
Gemini API rate limit (free tier)	Trung bình	Cache kết quả AI, chỉ gọi khi user chủ động yêu cầu
Upload ảnh chậm trên mạng yếu	Trung bình	Compress ảnh trước upload + hiển thị progress
Firestore cost nếu nhiều user	Thấp	Security rules chặt chẽ, index đúng field
Token FCM hết hạn	Thấp	Refresh token tự động qua <code>onNewToken()</code>

Điểm Khác Biệt Pocket So Với Locket

- 1. **AI Smart Caption (tiếng Việt)** — Locket chưa có tính năng này
- 2. **Ẩn số điện thoại** — Bảo vệ privacy người dùng tốt hơn
- 3. **Bản địa hóa hoàn toàn** — Giao diện, notification, caption đều tiếng Việt
- 4. **Auto-login** — Trải nghiệm mượt mà hơn, không phải đăng nhập lại mỗi lần mở app
- 5. **Chat tích hợp** — Nhắn tin trực tiếp với notification real-time